Intermediate (two-level logger) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/logging_practice.py`. Run it with `cd practice && python logging_practice.py`.

Used later by: the Real-world wiring exercise **copies** the `get_logger()` you write here into `practice/config_logging_wiring/logging_setup.py`, and the Build Task writes the same function again in `practice/build_task/logging_setup.py`. A copy, not an import — each of those folders stands alone. Keep the name and signature exactly as they are: `get_logger(name: str) -> logging.Logger`.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

A logger can have more than one **handler** attached, and each handler can have its own level. That's the whole trick here: one handler pointed at the screen, set to `INFO`; one handler pointed at a file, set to `DEBUG`. The logger itself just needs to be set to the *lowest* level of any of its handlers, or messages get filtered out before they even reach the handlers.

Think of it as two filters in a row: the logger's own level is the first gate ("do I even bother with this message?"), and each handler's level is a second gate ("does *this* handler want it?"). A message has to pass both gates to end up somewhere.

Here are the exact pieces, with the calls you'll actually need:

- `logging.getLogger(__name__)` — get your logger.
- `logging.StreamHandler()` — a handler that prints to the screen.
- `logging.FileHandler("app.log")` — a handler that writes to a file.
- `handler.setLevel(logging.INFO)` / `handler.setLevel(logging.DEBUG)` — each handler's own cutoff.
- `logger.addHandler(handler)` — attach a handler (call this twice, once per handler).

- `logger.setLevel(logging.DEBUG)` — the logger's own level must be at least as permissive as its most permissive handler.
- Wrapping all of this in a function named exactly `def get_logger(name: str) -> logging.Logger:` — keep that name, the Real-world wiring exercise and the Build Task both import `get_logger` by it.

Hint 2 — The plan, and almost the whole thing

```
define a function get_logger(name: str) -> logging.Logger:
    get a logger named after this module
    set the logger's own level to DEBUG (the lowest, most permissive)

    create a screen handler, set its level to INFO
    create a file handler pointed at "app.log", set its level to DEBUG

    attach both handlers to the logger
    return the logger

logger = get_logger(__name__)

log one DEBUG message   -> should appear in the file only
log one INFO message    -> should appear on screen AND in the file
log one WARNING message -> should appear on screen AND in the file
```

```
# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    # add the file handler yourself, the same way, then return logger

logger = get_logger(__name__)
# add the three log calls yourself — see the Solution if stuck
```

Finish the function, then compare against the Solution.

Full solution: Show me the solution